

Reporte

Universidad Nacional Autónoma de México

Modelado y Programación

Facultad de Ciencias

Jafet Felipe Flores



Resumen

Este proyecto trata de implementar y diseñar un chat a través de una comunicación cliente-servidor, el cual será desarrollado en el lenguaje de programación Go, tanto el cliente como el servidor. La comunicación se basa en un intercambio de mensajes JSON. El servidor se encargará de procesar las conexiones, que cada cliente se identifique antes de poder hacer cualquier otra cosa, así también como responder a los JSON recibidos según sea cada caso. Por la parte del cliente, es quien manda los mensajes al servidor para ser debidamente procesados. Se hace uso de la concurrencia para poder atender múltiples conexiones. Este programa tiene la finalidad de permitir la conexión de varios clientes, de manera que se puedan comunicar entre ellos, todo esto siguiendo un protocolo específico. Los saltos de línea son los que se tienen pensados que sean los que delimiten el final de un mensaje. En la implementación de este proyecto se hizo uso de la comunicación TCP y de las goroutines para atender conexiones concurrentemente.

Introducción

El principal problema que se busca resolver en este proyecto es poder tener un programa que permita la comunicación entre dos o más clientes por medio de un chat. Esto se hará mediante un sistema separado de la vista del cliente que tiene el objetivo de que el servidor se encargue de mantener los registros de clientes que se encuentran conectados, mientras que el cliente envía solicitudes y procesa cada mensaje recibido.

El alcance es local por el momento ya que solamente es posible establecer una comunicación entre varios clientes dentro de una misma red Wi-Fi. En este caso, ese sería una de las limitaciones que presenta nuestro programa, aunque no se necesita hacer gran cosa para poder chatear entre distintos clientes.

Los objetivos principales se presentan a continuación:

- Establecer una comunicación satisfactoria TCP entre varios clientes y el servidor
- Con base en el protocolo definir los mensajes JSON con respuestas y procesos
- Administrar clientes, sus estados y las salas que se crearán después
- Manejar posibles errores como de identificación o clientes inexistentes
- Organizar el código y asignar nombres coherentes a todo

Además, se usará el MVC (Modelo-Vista-Controlador) para poder más fácilmente organizar el trabajo que cada parte debe hacer. Cabe mencionar que este proyecto se hará con el paradigma de Programación Orientado a Objetos (POO).

Análisis y diseño

Funcionalidades básicas

Lo siguiente está dado por el protocolo.

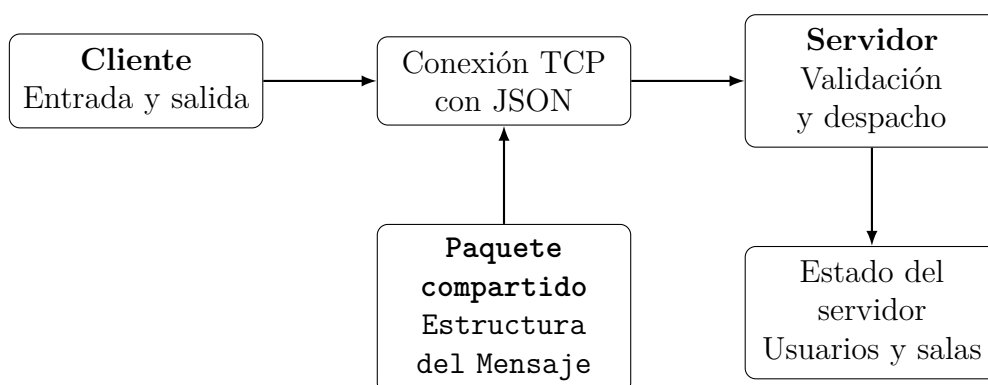
1. Identificar cada cliente con un nombre único máximo de 8 caracteres
2. Consultar que clientes están conectados y ver sus estados
3. Cambiar el estado de un cliente, con las opciones: **ACTIVE**, **AWAY** y **BUSY**
4. Enviar mensajes públicos y privados
5. Crear salas e invitar usuarios a unirse a salas, y pedir la lista de clientes
6. Enviar mensajes a los clientes de una sala y/o abandonarla
7. Notificar la desconexión, así también como la llegada de cada cliente

Mensajes JSON

Aquí tenemos una comunicación TCP, donde los mensajes están (deben estarlo) codificados como un JSON válido que está delimitado por un salto de línea. De nuevo, si la longitud del nombre de un usuario es mayor a 8 se harán las operaciones indicadas por el protocolo, como es que si ocurre un error de la falta de un argumento esperado después de un indicador de comando se debe mandar un mensaje JSON y desconectar al cliente.

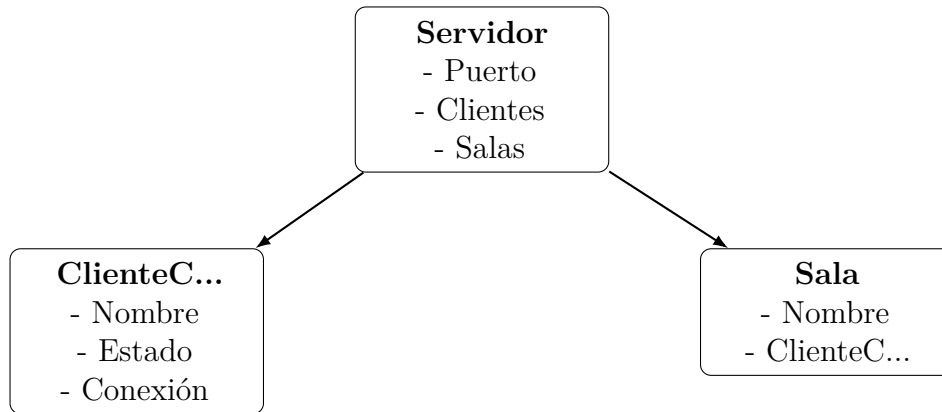
Estructura general

La arquitectura se divide en el cliente y el servidor (adicionalmente un paquete que se comparte entre ellos). En el cliente se interpretan los mensajes recibidos por parte del usuario y se envían mensajes. El servidor es la parte que se encarga de procesar múltiples conexiones, y una vez que se valida la identificación del cliente procede a recibir los mensajes del cliente, actualizar estados, mandar mensajes acorde a la petición del cliente, etc. El paquete compartido define la estructura de los mensajes.



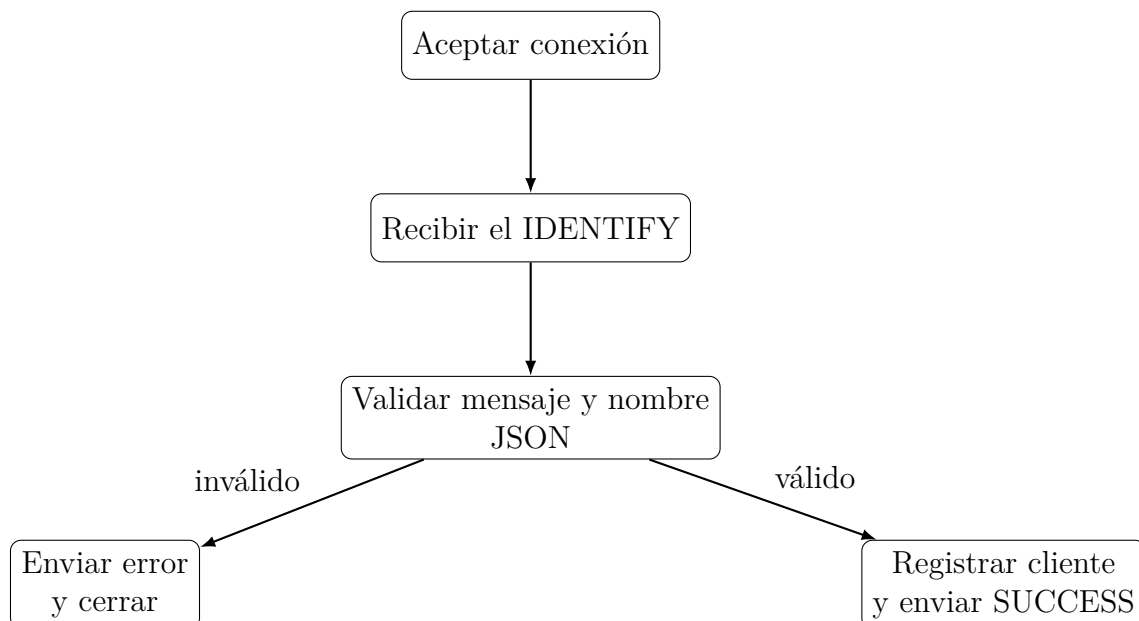
Estructura del mensaje

La estructura del **Mensaje** representa los campos del protocolo, como el tipo de operación (**Operation**), el nombre de usuario (**Username**), el resultado (**Result**) y los datos adicionales (**Extra**). El servidor básicamente maneja un map de **ClienteConectado** y otro de salas. Cada uno de los clientes tiene su nombre, estado y conexión. Y cada sala mantiene su nombre y los clientes que pertenecen a esa sala.



Proceso de identificación

Cuando se establece una conexión, el cliente en automático envía una solicitud de identificación. Luego de esto, el servidor comprueba si el JSON recibido es válido y que lo primero que se haga sea identificarse con el tipo **IDENTIFY**, que el nombre no sea una cadena vacía, y también se cubre el caso cuando el nombre con el que se intenta identificar se encuentra en uso por otro cliente. En el caso de que todo salga bien, se registra al cliente y se responde con un mensaje de éxito. De lo contrario, se envía el error correspondiente y se cierra la conexión con el servidor.



Comunicación cliente-servidor

Se muestra en el siguiente esquema el intercambio de mensajes entre el servidor y el cliente en el proceso de identificación, donde se sigue el algoritmo de solicitar algo y obtener una respuesta dependiendo del mensaje.



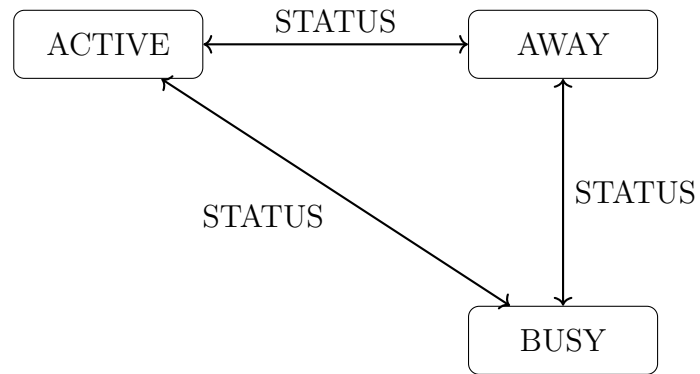
Tarjetas CRC (Clase-Responsabilidad-Colaborador)

He aquí están las tarjetas CRC que nos permiten visualizar las responsabilidades de cada una de las clases, así como las clases con las que están relacionadas (colaborando).

Clase	Responsabilidades	Colaboradores
Servidor	Aceptar conexiones. Validar mensajes. Administrar usuarios y también salas. Mandar mensajes	ClienteConectado, Sala, Mensaje
ClienteConectado	Mantener el nombre, estado y conexión del cliente	Servidor
Sala	Mantener a los integrantes, saber quiénes han sido invitados	Servidor y ClienteConectado
Cliente	Conectarse al servidor. Enviar comandos, recibir y mostrar mensajes	Servidor y Mensaje
Mensaje	Representar las solicitudes, respuestas y notificaciones dadas en el protocolo	Cliente y Servidor

Estados del cliente

Como estado por default de un cliente (que ya se ha identificado) es **ACTIVE**. Y puede cambiar a **AWAY** o **BUSY** por medio de una solicitud válida, pues si se intenta solicitar por ejemplo un estado **BORED** que no está entre las opciones de un estado se desconectará al cliente luego de dar el aviso de mensaje inválido. Si se efectúa el cambio de estado, se debe de notificar a los demás clientes.



Implementación

Tecnologías usadas y organización

El programa lo he desarrollado en el lenguaje de programación Go, donde he hecho uso del paquete `net` para las conexiones TCP, del paquete `encoding/json` para codificar y decodificar los mensajes, de las goroutines para poder atender conexiones concurrentemente. En este proyecto se separa el código del servidor, del cliente y de lo que se comparte entre ellos para tener una mejor visibilidad de lo que está pasando en el programa. Lo siguiente es un esquema donde se puede apreciar la atención de conexiones dentro del programa del servidor.

Listing 1: Esquema simplificado de atención de conexiones

```
listener, err := net.Listen("tcp", direccion)
if err != nil {
    return err
}
defer listener.Close()

for {
    conn, err := listener.Accept()
    if err != nil {
        continue
    }
    go servidor.AtiendeConexion(conn)
}
```

Manejo de mensajes

Cuando se representa cada mensaje, se hace mediante una estructura compartida transmitiéndose como JSON con al menos un campo. El campo `type` nos dice la clase de mensaje recibido, el campo `Operation` nos dice la acción que se intentó realizar, el campo `Username` nos dice el nombre de usuario, y así con los demás campos establecidos por el protocolo. Lo que hacemos cuando uno o varios campos no son necesarios que aparezcan, es desde el paquete compartido de Mensaje, etiquetarlos con `omitempty` y así se omite cuando el campo es vacío.

Operaciones principales

Operación	Descripción
IDENTIFY	Identifica a un cliente (automático al conectarse)
STATUS	Modifica el estado del cliente
USERS	Solicita la lista de clientes conectados
TEXT / PUBLIC_TEXT	Envía mensajes privados / públicos
NEW_ROOM	Crea una sala para chatear
INVITE / JOIN_ROOM	Invita clientes a una sala / permite su ingreso a ella
ROOM_USERS / ROOM_TEXT	Consulta integrantes / envía mensajes a una sala
LEAVE_ROOM / DISCONNECT	Deja una sala / Desconecta al cliente

Tabla 1: Comandos y argumentos de los mensajes del protocolo

Operación	Uso	Ejemplo
IDENTIFY	Automático al conectarse	<code>{"type": "IDENTIFY", "username": "Juan"}</code>
STATUS	STATUS <estado>	STATUS BUSY
USERS	USERS	USERS
PUBLIC_TEXT	<texto>	Hola a todos
TEXT	TEXT <username> <mensaje>	TEXT Kay Hola, ¿cómo estás?
NEW_ROOM	NEW_ROOM <roomname>	NEW_ROOM Tareas
INVITE	INVITE <roomname> <usernames>	INVITE Tareas Kay
JOIN_ROOM	JOIN_ROOM <roomname>	JOIN_ROOM Tareas
ROOM_USERS	ROOM_USERS <roomname>	ROOM_USERS Tareas
ROOM_TEXT	ROOM_TEXT <roomname> <mensaje>	ROOM_TEXT Tareas Me pasas la tarea?
LEAVE_ROOM	LEAVE_ROOM <roomname>	LEAVE_ROOM Tareas
DISCONNECT	DISCONNECT	DISCONNECT

Validación y manejo de posibles errores

El servidor rechaza mensajes JSON inválidos (mal contruidos), solicitudes distintas de la identificación como primer paso, nombres vacíos o repetidos y operaciones con parámetros incorrectos. También se comprueba la existencia de los clientes a los que se les efectúa una determinada acción, como invitar a una sala, así como la comprobación de la pertenencia del cliente a la sala a la que quiere invitar a otro cliente. Así con los casos que lo requieran.

Requerimientos para el programa

Solamente ejecutar los dos archivos **Servidor** y **Cliente**, pero comenzando con el del servidor. Se debe hacer `./Servidor` y luego con el del cliente `./Cliente IP:HOST <nombre>`, suponiendo que están conectados a la misma red Wi-Fi. Solamente recordando que el nombre debe ser a lo más de 8 caracteres de longitud, y dentro del programa, al crear una sala su nombre debe ser máximo de 16 caracteres.

Conclusiones

Después del desarrollo de este proyecto, aprendí con más claridad cómo es que funciona la comunicación TCP, en este caso del cliente-servidor. Además, pude aprender (no en su totalidad) como programar en Go, y una técnica usando `net.Pipe()` en go, que es básicamente simular una conexión entre dos extremos en memoria, esto me ayudó a realizar pequeñas pruebas sin necesidad de establecer una conexión real. Lo que se me dificultó fue entender como era esa comunicación que al final de cuentas era lo principal en este proyecto.

Veo un área de mejora en mis comentarios dentro del proyecto, y también en como nombro tanto las variables como las funciones, además de organizar mis ideas. Si en algún futuro pudiera hacer de este proyecto algo mejor, lo que haría sería darle más alcance y ponerle al menos una interfaz bonita. Lo que me ayudó a visualizar el funcionamiento de este proyecto fue usar el paradigma de POO y el MVC, así también como los distintos diagramas y tarjetas CRC. Otra cosa que se me dificultó fue cubrir todos los posibles casos dados implícita y explícitamente en el protocolo, pues al cubrir unos salían otros nuevos que no había contemplado.